Homework 6

March 8, 2021

Chun-Yuan (Scott) Chiu

chunyuac@andrew.cmu.edu

This is starter code for homework 6. Most of the code won't run as is. You need to complete the missing parts of the functions.

[15]:
```
import numpy as np
import sklearn as sk
import matplotlib
import matplotlib.pyplot as plt
import scipy.stats
from sklearn.linear_model import LinearRegression
import pandas as pd
```

[18]:
```
matplotlib.rcParams['figure.figsize'] = (12,8)
```

Problem 1

Part a

Fill in the missing parts of the classes below, marked with TODO.

[2]:
```
# This class represents your loss function, (1/2) sum((y-yhat)^2).
# Note: we are including a 1/2, which we don't always do, so that your␣
 ↪derivatives will be particularly nice.
# The functions should operate on vectors y and yhat
class Loss(object):
    def __init__(self):
        # These fields will store the last seen y and yhat
        # The stored values will be used by backward() to compute the gradient␣
 ↪of this function
        # at the last evaluated point
        self.last_y = 0
```

```
        self.last_yhat = 0

    # This function returns the value of the loss
    def forward(self, y, yhat):
        # Store the new values for use in the gradient
        self.last_y = y
        self.last_yhat = yhat
        return 0.5*((self.last_y - self.last_yhat)**2).sum()

    # This function returns the gradient of the loss: y - yhat
    # Important: This must be an (n,1) matrix.  Not (n,).
    def backward(self):
        # Grab the length of y for shape checking
        n = len(self.last_y)
        return_value = -(self.last_y - self.last_yhat)

        # Let's check the shape of the return value to keep you sane
        assert return_value.shape == (n,1), "Loss backward returns wrong shape"
        return return_value

# This class represents your linear function, beta -> X\beta.  Should work for␣
 ↪an n by p matrix X.
class LinearFunction(object):
    # Linear function is initialized with a dimension p, and an optional␣
 ↪starting value beta
    def __init__(self, p, beta=None):
        self.p = p
        if beta is None:
            self.beta = np.zeros((p,1))
        else:
            #We make sure that beta starts with the right shape, even if you␣
 ↪send in the wrong shape
            self.beta = beta.reshape((p,1))
        #Store the last X seen by this function, initialize to 0
        self.last_X = 0

    # This function returns the value of the linear function at an X and the␣
 ↪current beta: X*beta
    def forward(self, X):
        # Record the new X for use later in the gradient
        n,p = X.shape
        self.last_X = X

        # TODO: You need to compute the value of your linear function: X*beta␣
 ↪(matrix multiplied)
        # Make sure that your return value is n by 1
        return_value = self.last_X @ self.beta
```

```
        assert return_value.shape == (n,1), "Linear forward returns wrong shape"
        return return_value

    def backward(self):
        # TODO: You need to compute the gradient of the linear function with␣
↪respect to beta. (X^T)
        # Make sure that your return value is p by n
        n,p = self.last_X.shape

        return_value = self.last_X.T

        # Let's check the shape of the return value to keep you sane
        assert return_value.shape == (p,n), "Loss backward returns wrong shape"
        return return_value

    # This is just here to make your future updates ever so slightly easier. ␣
↪This way your gradient updates
    # will just be a function call, and the shape will be confirmed correct
    def beta_shift(self, shift):
        assert self.beta.shape == shift.shape
        self.beta = self.beta + shift
```

That's all there is to it! We can link them together using the chain rule, and then solve linear regression by coordinate descent!

First, we generate some trial data to experiment with:

[3]:
```
n = 100
p = 3
beta = np.array([3,1,4]).reshape((-1,1))
np.random.seed(1)
X = scipy.stats.norm.rvs(size=n*p).reshape((n,p))
y = np.dot(X,beta) + scipy.stats.norm.rvs(size=n, scale=0.1).reshape((n,1))
```

[4]:
```
# Check that the coefficient is about what we think it should be.
# Hopefully you will soon have an implementation that agrees with this.
fit = LinearRegression().fit(X,y)
fit.coef_
```

[4]:
```
array([[3.00263927, 0.98650739, 3.99947187]])
```

Check 1a

Execute this code to check your implementation. This should execute without errors and without triggering the assertions about return value shapes. Summaries are computed of the gradients to

verify you're getting the right numbers.

[5]:
```
# We can initialize the pieces of our solver. Right now, beta is initialized to␣
 ↪a vector of zeros
linear = LinearFunction(p)
loss = Loss()

# Computes the squared error loss for your current X, evaluated with beta = 0
print(loss.forward(y, linear.forward(X))) # Should be approximately 1087.15

# Compute the gradient of the linear piece and summarize by the mean for␣
 ↪checking
print(np.mean(linear.backward(), axis=1))
# Should be [0.0179277 , 0.08671813, 0.11855369]

#Compute the gradient of the loss with respect to yhat and summarize by mean
print(np.mean(loss.backward())) # Should be approximately -0.613
```

```
1087.1524810665376
[0.0179277  0.08671813 0.11855369]
-0.6126797649214445
```

Part b

Put it all together into what will become one iteration of gradient descent. The end goal here is to produce a gradient of the loss with respect to beta, which should have shape p x 1

[6]:
```
# TODO: Fill in each of the missing lines
# Compute the loss for linear function evaluated at current X
## TODO ##

# Compute the gradient of the loss with respect to beta.
# This should be (gradient from linear) * (gradient from loss)
grad = linear.backward() @ loss.backward()

# Print out your final gradient of the loss with respect to beta for the first␣
 ↪step from initialization above.
print(grad)
```

```
[[-283.8105542 ]
 [ -39.20095874]
 [-320.63943886]]
```

Part c

Now you will actually write gradient descent and solve the problem!

[7]:
```
# Number of steps of gradient descent
reps = 100
# Learning rate (in practice, often decreases with iteration)
eta = 0.01

# Initialize the pieces of our solver. beta is initialized to a vector of zeros
linear = LinearFunction(p)
loss = Loss()

# Let's keep track of the loss at each step, so you can visualize your␣
 ↪optimization later
losses = np.zeros(reps)

grads = []

for i in range(reps):
    # Compute and store your current loss
    losses[i] = loss.forward(y, linear.forward(X))

    # Compute the gradient of loss with respect to beta
    grad = linear.backward() @ loss.backward()
    grads.append(grad)

    # Adjust beta (using beta_shift) by -grad*eta
    linear.beta_shift(-grad*eta)
```

Part d

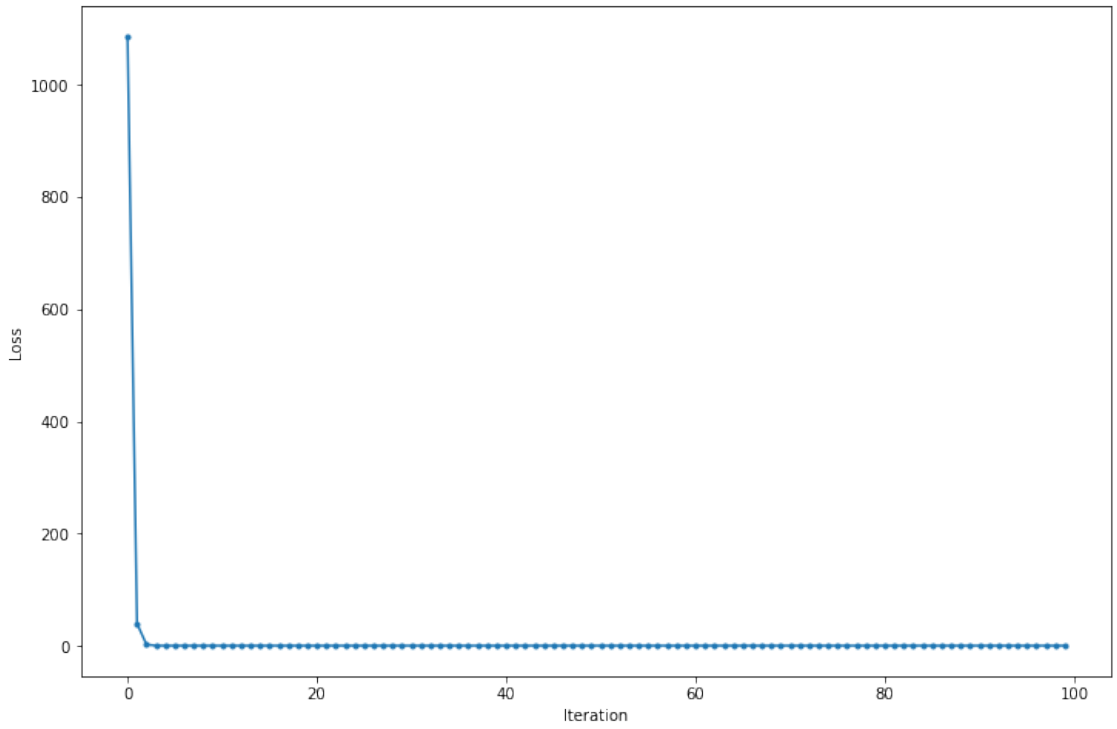
The training result β, as printed out here, is very close to the benchmark $(3, 1, 4)^T$.

[8]:
```
linear.beta
```

[8]:
```
array([[3.00261269],
       [0.98641329],
       [3.99934409]])
```

The loss decreases smoothly and rapidly towards zero and converges within the first few iterations.

[11]:
```
from pandas import Series

ax = Series(losses).plot(style='.-')
ax.set(xlabel='Iteration', ylabel='Loss')
pass
```



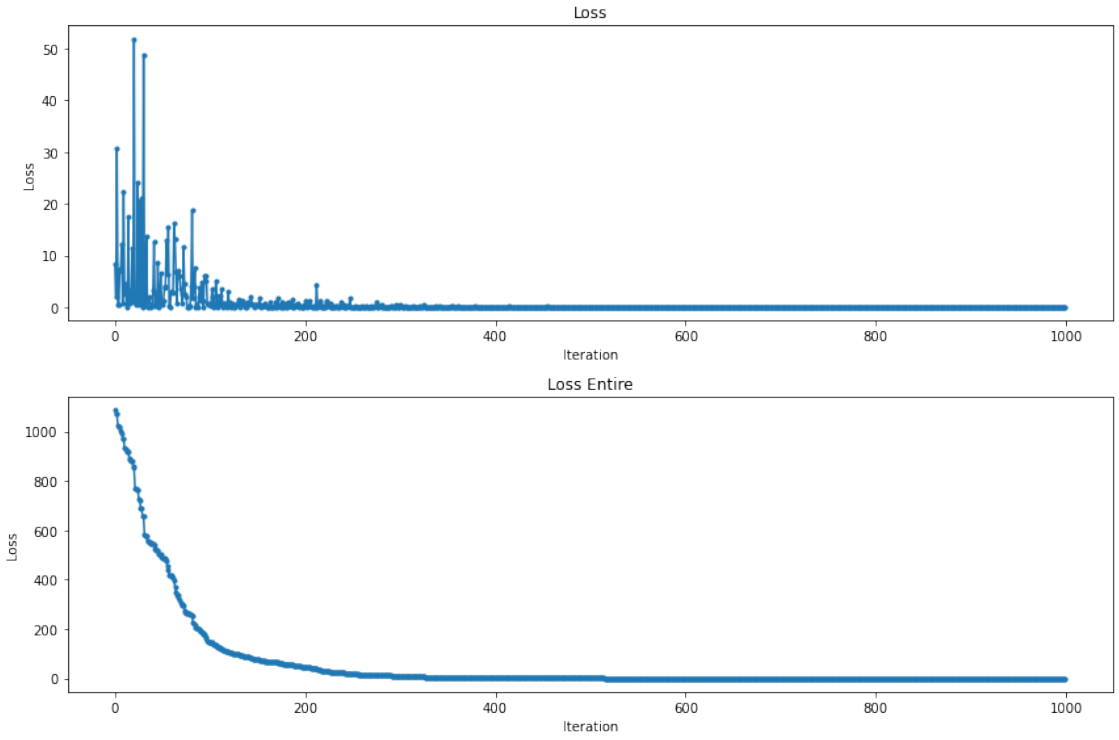
Part e

After 1000 iterations, the estimated β is quite close to the benchmark $(3, 1, 4)^T$. The loss converges to zero but slowly and noisily. Nevertheless, when dealing with huge data sets, only a limited subset of the data can be put into computer memory for training at each step, that is why the stochastic gradient descent, despite the slowdown and noises in convergence, can be quite useful.

[20]:
```
# Number of steps of gradient descent.  We need more for the stochastic version
reps = 1000
# Learning rate (in practice, often decreases with iteration)
eta = 0.01

# Initialize the pieces of our solver. beta is initialized to a vector of zeros
linear = LinearFunction(p)
loss = Loss()

# Let's keep track of the loss at each step, so you can visualize your␣
 ↪optimization later.
# We will track two losses: The loss on your single sampled point, and the loss␣
 ↪on your whole sample.
losses = np.zeros(reps)
losses_entire = np.zeros(reps)
```

```
for i in range(reps):
    # Pick a single observation to show the optimizer this time
    k = np.random.randint(y.shape[0])

    # Compute and store your current loss, only for the kth row of X and kth y␣
 ↪element!
    # Note: when subsetting X, use X[np.newaxis, k,:].  This keeps the␣
 ↪resulting row
    # two dimensional, rather than dropping to a 1d vector.
    losses[i] = loss.forward(y[np.newaxis, k], linear.forward(X[np.newaxis, k,:
 ↪]))

    # Compute the gradient of loss with respect to beta
    grad = linear.backward() @ loss.backward()

    # Compute losses on the whole sample (you wouldn't usually actually do␣
 ↪this, too expensive)
    losses_entire[i] = loss.forward(y, linear.forward(X))

    # Adjust beta (using beta_shift) by -grad*eta
    linear.beta_shift(-grad*eta)


fig, (ax1, ax2) = plt.subplots(2, 1)
Series(losses).plot(style='.-', ax=ax1)
Series(losses_entire).plot(style='.-', ax=ax2)
ax1.set(xlabel='Iteration', ylabel='Loss', title='Loss')
ax2.set(xlabel='Iteration', ylabel='Loss', title='Loss Entire')
plt.tight_layout()

linear.beta
```

[20]:
```
array([[3.00761461],
       [0.98015708],
       [3.99774098]])
```



Problem 2

[10]:
```
import numpy as np
import scipy.stats
import torch
import torch.autograd
import torch.nn.functional as F
```

[11]:
```
# Same data as before
n = 100
p = 3
beta = np.array([3,1,4]).reshape((-1,1))
np.random.seed(1)
X = scipy.stats.norm.rvs(size=n*p).reshape((n,p))
y = np.dot(X,beta) + scipy.stats.norm.rvs(size=n, scale=0.1).reshape((n,1))
```

[12]:
```
# Convert your data X, y to tensors, using torch.Tensor
X = torch.Tensor(X)
y = torch.Tensor(y)
```

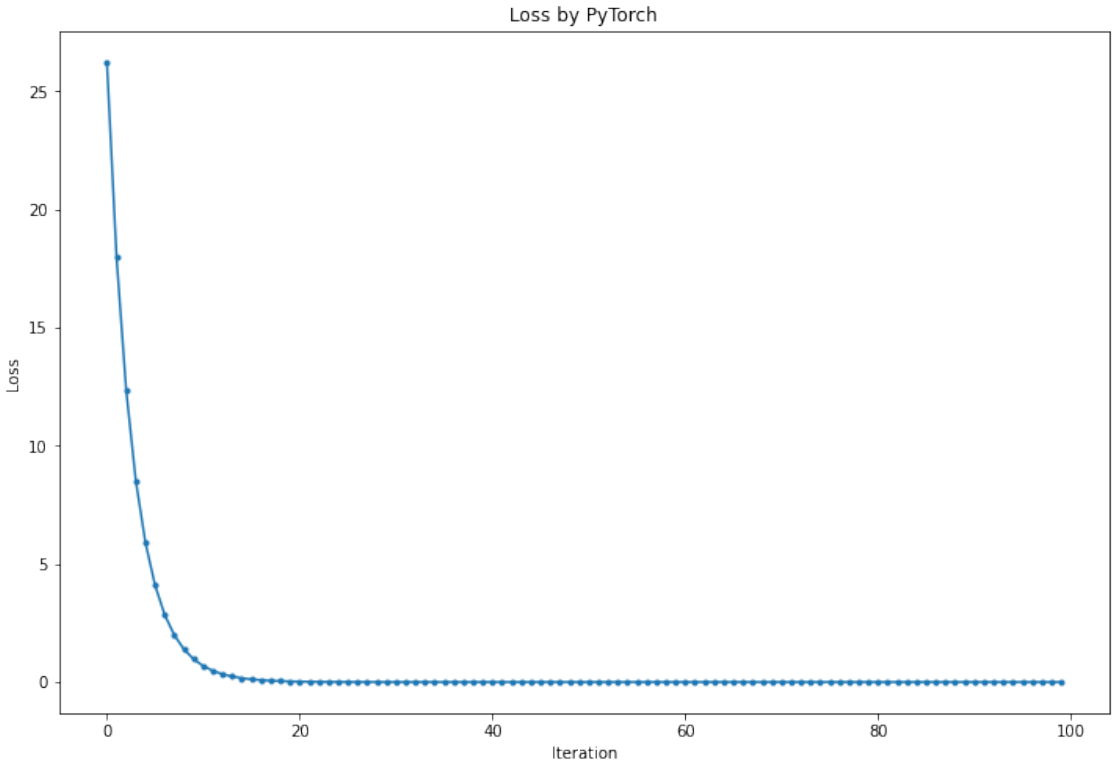
```
# Convert those tensors to variables, so that torch will track gradients.  Use␣
 ↪torch.autograd.Variable
X = torch.autograd.Variable(X)
y = torch.autograd.Variable(y)

# Make a linear function using torch.nn.Linear
# Set the bias to False turns off intercept. Not usally desirable, but want to␣
 ↪match our problem 1 code.
fc = torch.nn.Linear(p, 1, bias=False)
```

[13]:
```
reps = 100
losses = np.zeros(reps)
for i in range(reps):

    # Reset gradients
    fc.zero_grad()

    # Forward pass: Compute the mse_loss between fc(X) and y, using F.
 ↪mse_loss(guess, truth)
    # Store the output in losses[i]
    output = F.mse_loss(fc(X), y) ## TODO ##
    losses[i] = output

    # Compute all the gradients
    output.backward()

    # Apply gradients.  In practice, this can be carried out by built in␣
 ↪functions
    # as well, like torch.optim.SGD
    for param in fc.parameters():
        param.data.add_(-0.1 * param.grad.data)
```

The resulting β is close to the benchmark as expected. Below is a plot of the losses at each iteration.

[21]:
```
from pandas import Series

ax = Series(losses).plot(style='.-')
ax.set(xlabel='Iteration', ylabel='Loss', title='Loss by PyTorch')
param
```

[21]:
```
Parameter containing:
tensor([[3.0026, 0.9864, 3.9993]], requires_grad=True)
```


Loss by PyTorch
Loss
Iteration
25
20
15
10
5
0
0
20
40
60
80
100